

LAPORAN PRAKTIKUM STRUCTUR DATA

PEKAN 2

QUEUE



Disusun Oleh :

UMAR ABDUL AZIZ

NIM 2511532012

DOSEN PENGAMPU :

Wahyudi Dr S,T.M.T

DEPARTMENT INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG, 30 APRIL 2026

KATA PENGHANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum yang berjudul Praktikum dengan judul “**Queue**” ini dengan baik dan tepat waktu..

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan. Selain itu, laporan ini juga bertujuan untuk menambah pemahaman serta wawasan mengenai materi yang telah dipelajari selama kegiatan praktikum berlangsung. Dalam proses penyusunan laporan ini, saya memperoleh banyak pengalaman dan pengetahuan yang bermanfaat, baik secara teori maupun praktik.

Saya menyadari bahwa dalam penyusunan laporan ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, saya mengucapkan terima kasih kepada dosen pembimbing, asisten praktikum, serta teman-teman yang telah memberikan dukungan, bimbingan, dan bantuan sehingga laporan ini dapat terselesaikan.

Saya juga menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi pembaca serta menjadi referensi yang berguna.

Akhir kata saya ucapkan teriima kasih.

Padang 30 April 2026

Penulis

DAFTAR ISI

KATA PENGHANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
A. LATAR BELAKANG.....	4
B. RUMUSAN MASALAH	4
C. TUJUAN.....	4
D. PERSYARATAN PRAKTIKUM	4
BAB II PEMBAHASAN.....	5
A. PENGERTIAN	5
B. PEMBAHASAN.....	5
BAB III KESIMPULAN.....	11

BAB I

PENDAHULULAN

A. LATAR BELAKANG

Dalam pemrograman, struktur data digunakan untuk mengelola data secara efisien, salah satunya adalah queue atau antrian. Queue menggunakan prinsip FIFO (First In First Out), di mana data yang masuk lebih dulu akan keluar lebih dulu. Konsep ini banyak diterapkan dalam kehidupan sehari-hari dan sistem computer

Oleh karena itu, pemahaman tentang queue penting untuk membantu dalam pengolahan data secara teratur. Praktikum ini bertujuan agar mahasiswa dapat memahami dan mengimplementasikan queue dalam pemrograman.

B. RUMUSAN MASALAH

1. Bagaimana konsep dasar struktur data queue dengan prinsip FIFO (First In First Out)?
2. Bagaimana cara mengimplementasikan operasi dasar queue dalam bahasa pemrograman Java?
3. Bagaimana cara mengelola dan memanipulasi data antrian menggunakan program queue?

C. TUJUAN

1. Mengetahui konsep dasar struktur data queue dengan prinsip FIFO (First In First Out).
2. Mampu mengimplementasikan operasi dasar queue dalam bahasa pemrograman Java.
3. Mampu mengelola dan memanipulasi data antrian menggunakan program queue.

D. PERSYARATAN PRAKTIKUM

- Telah mengikuti perkuliahan teori Struktur data sebagai dasar pemahaman.
- Membawa perlengkapan yang diperlukan, antara lain laptop atau computer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Enviroment (IDE) yang direkomendasikan.
- Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
- Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
- Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

BAB II

PEMBAHASAN

A. PENGERTIAN

Queue adalah salah satu jenis struktur data dalam pemrograman yang digunakan untuk menyimpan dan mengelola data secara berurutan berdasarkan prinsip FIFO (First In First Out). Artinya, data yang pertama kali dimasukkan ke dalam antrian akan menjadi data pertama yang dikeluarkan. Hal ini membuat queue berbeda dengan struktur data stack yang menggunakan prinsip LIFO (Last In First Out).

Pada struktur data queue, terdapat dua bagian utama yaitu front dan rear. Bagian front berfungsi sebagai tempat penghapusan data, sedangkan rear digunakan sebagai tempat penambahan data baru. Proses penambahan data disebut enqueue, sedangkan proses penghapusan data disebut dequeue. Kedua operasi ini merupakan operasi dasar yang selalu ada dalam implementasi queue.

Queue banyak digunakan dalam berbagai bidang, terutama pada sistem komputer. Contohnya adalah pada sistem operasi untuk mengatur antrian proses, pengelolaan data pada printer (print queue), serta dalam sistem jaringan untuk mengatur paket data yang masuk dan keluar. Selain itu, konsep queue juga sering ditemukan dalam kehidupan sehari-hari, seperti antrian di kasir, loket pelayanan, dan pembelian tiket.

Dengan memahami konsep queue, kita dapat mengelola data secara lebih terstruktur dan efisien, terutama pada sistem yang membutuhkan pengolahan data berdasarkan urutan kedatangan..

B. PEMBAHASAN

a) QueueArray

```
package Pekan4_2511532012;

public class QueueArray_2511532012 {
    int front_2012, rear_2012, size_2012;
    int capacity_2012;
    int array_2012[];

    public QueueArray_2511532012 (int capacity_2012) {
        this.capacity_2012 = capacity_2012;
    }
}
```

```

        front_2012 = this.size_2012 = 0;
        rear_2012 = capacity_2012 - 1;
        array_2012 = new int[ this.capacity_2012];

    }

    boolean isFull_2511532012 (QueueArray_2511532012 queue) {
        return (queue.size_2012 == queue.capacity_2012);
    }

    boolean isEmpty_2511532012 (QueueArray_2511532012 queue) {
        return (queue.size_2012 == 0);
    }

    void enqueue_2511532012 (int item) {
        if (isFull_2511532012(this))
            return;
        this.rear_2012 = (this.rear_2012 + 1) % this.capacity_2012;
        this.array_2012 [this.rear_2012] = item;
        this.size_2012 = this.size_2012 + 1 ;
        System.out.println(item + " enqueued to queue");
    }

    int dequeue_2511532012 () {
        if (isEmpty_2511532012(this))
            return Integer.MIN_VALUE;
        int item = this.array_2012[this.front_2012];
        this.front_2012 = (this.front_2012 + 1) % this.capacity_2012;
        this.size_2012 = this.size_2012 - 1;
        return item;
    }

    int front_2511532012 () {
        if (isEmpty_2511532012(this))
            return Integer.MIN_VALUE;

        return this.array_2012[this.front_2012];
    }

    int rear_2511532012 () {
        if (isEmpty_2511532012(this))
            return Integer.MIN_VALUE;
        return this.array_2012[this.rear_2012];
    }

    // mencetak elemen antrian
    void display_2511532012() {
        int i;
        if (front_2012 == rear_2012) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }
        //kunjungi dari belakang dan cetak
        for (i = front_2012 ; i < rear_2012 ; i++) {
            System.out.printf(" %d <-- ", array_2012[i]);
        }
    }

```

```

        return;
    }
}

```

Program ini merupakan implementasi struktur data queue (antrian) menggunakan array bertipe data integer dalam bahasa Java. Queue pada program ini menggunakan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Di dalam program terdapat beberapa variabel utama, yaitu front sebagai penunjuk elemen terdepan, rear sebagai penunjuk elemen terakhir, size untuk menyimpan jumlah elemen saat ini, capacity sebagai kapasitas maksimum queue, serta array sebagai tempat penyimpanan data.

Program ini juga menggunakan konsep circular queue, yaitu ketika indeks array mencapai batas akhir maka akan kembali ke awal menggunakan operasi modulus. Hal ini bertujuan agar penggunaan memori lebih efisien. Fungsi isFull digunakan untuk mengecek apakah antrian sudah penuh, sedangkan isEmpty digunakan untuk mengecek apakah antrian kosong.

Operasi enqueue berfungsi untuk menambahkan data ke dalam antrian pada posisi belakang (rear), sedangkan dequeue digunakan untuk menghapus data dari bagian depan (front) dan mengembalikan nilai yang dihapus. Selain itu, terdapat fungsi front untuk melihat elemen paling depan tanpa menghapusnya, serta fungsi rear untuk melihat elemen paling belakang.

Fungsi display digunakan untuk menampilkan seluruh isi antrian. Program akan mencetak elemen dari posisi front hingga rear. Namun, jika antrian kosong maka akan menampilkan pesan bahwa antrian kosong. Secara keseluruhan, program ini menunjukkan implementasi dasar struktur data queue beserta operasi-operasinya dalam bentuk program sederhana.

b) QueueArrayDriver

```

package Pekan4_2511532012;

public class QueueArrayDriver_2511532012 {
    public static void main(String [] args) {
        QueueArray_2511532012 queue_2511532012 = new
        QueueArray_2511532012(1000);
        queue_2511532012.enqueue_2511532012(10);
        queue_2511532012.enqueue_2511532012(20);
        queue_2511532012.enqueue_2511532012(30);
        queue_2511532012.enqueue_2511532012(40);
        System.out.println("item di depan " +
        queue_2511532012.front_2511532012());
        System.out.println("item di belakang " +
        queue_2511532012.rear_2511532012());
        System.out.println("tampilkan queue");
        queue_2511532012.display_2511532012();
        System.out.println();
        System.out.println(queue_2511532012.dequeue_2511532012() + "
        di hapus dari queue");
        System.out.println("item di depan " +
        queue_2511532012.front_2511532012());
        System.out.println("item di belakang " +
        queue_2511532012.rear_2511532012());
        System.out.println("tampilkan queue setelah satu data di
        hapus");
        queue_2511532012.display_2511532012();
    }
}

```

Program ini merupakan driver class yang digunakan untuk menjalankan dan menguji implementasi struktur data queue yang telah dibuat sebelumnya. Pada program ini, dibuat objek queue dengan kapasitas 1000 menggunakan class QueueArray. Selanjutnya dilakukan beberapa operasi dasar queue, yaitu menambahkan data (enqueue), menghapus data (dequeue), serta menampilkan isi queue.

Awalnya, program menambahkan beberapa elemen ke dalam queue yaitu 10, 20, 30, dan 40 menggunakan fungsi enqueue. Setelah itu, program menampilkan elemen yang berada di posisi depan (front) dan belakang (rear) untuk mengetahui kondisi queue saat ini. Fungsi display digunakan untuk menampilkan seluruh isi queue yang telah dimasukkan.

Kemudian dilakukan operasi penghapusan data menggunakan dequeue, sehingga elemen yang berada di depan akan dihapus dari antrian. Setelah proses penghapusan, program kembali menampilkan elemen terdepan, elemen terakhir, serta isi queue yang sudah diperbarui. Dengan demikian, program ini berfungsi untuk memperlihatkan cara kerja queue secara langsung melalui operasi dasar seperti penambahan, penghapusan, dan penampilan data

c) QueueLinkedList

```
package Pekan4_2511532012;
import java.util.LinkedList;
import java.util.Queue;

public class QueueLinkedList_2511532012 {
    public static void main (String [] args) {
        Queue<Integer> q_2012 = new LinkedList<>();
        // tambah elemen {0, 1, 2, 3, 4, 5 } ke antrian
        for (int i = 0 ; i < 6 ; i ++ )
            q_2012.add(i);

        // menampilkann isi antrian
        System.out.println("Elemen antrian " + q_2012);

        //untuk menghapus kepala antrian
        int hapus_2012 = q_2012.remove();
        System.out.println("hapus elemen = " + hapus_2012);
        System.out.println(q_2012);

        //untuk melihat antrian terdepan
        int depan_2012 = q_2012.peek();
        System.out.println("kelapa antrian = " + depan_2012);

        int banyak_2012 = q_2012.size();
        System.out.println("Size antrian = " + banyak_2012);
    }
}
```

Program ini merupakan implementasi struktur data queue (antrian) menggunakan LinkedList melalui interface Queue dalam bahasa Java. Queue menerapkan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Pada program, queue dibuat untuk menyimpan data bertipe integer, kemudian elemen 0 sampai 5 ditambahkan ke dalam antrian menggunakan perulangan dan method add(). Setelah itu, isi antrian ditampilkan untuk menunjukkan kondisi awal queue.

Selanjutnya, program melakukan operasi dasar pada queue, yaitu remove() untuk menghapus elemen paling depan, peek() untuk melihat elemen terdepan tanpa menghapusnya, dan size()

untuk menghitung jumlah elemen yang tersisa dalam antrian. Hasil dari setiap operasi ditampilkan ke layar sehingga pengguna dapat melihat perubahan isi queue setelah proses penghapusan. Secara keseluruhan, program ini menunjukkan cara kerja dasar struktur data queue beserta operasi-operasi utamanya menggunakan LinkedList dalam Java.

d) ReverseData

```
package Pekan4_2511532012;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
public class ReverseData_2511532012 {

    public static void main(String[] args) {
        Queue<Integer> q_2012 = new LinkedList<Integer>();
        q_2012.add(1);
        q_2012.add(2);
        q_2012.add(3);
        System.out.println("sebelum revers " + q_2012);
        Stack<Integer> s_2012 = new Stack<Integer> ();
        while (!q_2012.isEmpty()) { // Q -> S
            s_2012.push(q_2012.remove());
        }
        while (!s_2012.isEmpty()) {
            q_2012.add(s_2012.pop());
        }
        System.out.println("sesudah revers = " + q_2012);
    }
}
```

Program ini merupakan implementasi pembalikan urutan data (reverse queue) dalam struktur data queue menggunakan bantuan stack pada bahasa Java. Queue dibuat menggunakan LinkedList dan diisi dengan elemen 1, 2, dan 3. Sebelum proses pembalikan dilakukan, isi queue ditampilkan terlebih dahulu untuk menunjukkan urutan awal data. Program memanfaatkan prinsip FIFO pada queue dan LIFO (Last In First Out) pada stack agar urutan elemen dapat dibalik.

Proses pembalikan dilakukan dengan memindahkan seluruh elemen dari queue ke stack menggunakan perulangan while, sehingga urutan data di dalam stack menjadi terbalik. Setelah queue kosong, seluruh elemen dalam stack dipindahkan kembali ke queue. Karena stack mengeluarkan elemen terakhir yang masuk terlebih dahulu, maka urutan data pada queue menjadi terbalik dari semula. Hasil akhirnya ditampilkan ke layar sehingga queue yang awalnya berisi [1, 2, 3] berubah menjadi [3, 2, 1]. Secara keseluruhan, program ini menunjukkan cara membalik isi queue menggunakan struktur data stack sebagai media bantu.

e) IterasiQueue

```
package Pekan4_2511532012;
import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;
public class IterasiQueue_2511532012 {
```

```

public static void main(String[] args) {
    Queue<String> q_2012 = new LinkedList<>() ;

    q_2012.add("praktikum");
    q_2012.add("struktur");
    q_2012.add("Data");
    q_2012.add("Dan");
    q_2012.add("Algoritma");

    Iterator<String> iterator = q_2012.iterator();
    while (iterator.hasNext()) {
        System.out.print(iterator.next() + " ");
    }
}

```

Program ini merupakan implementasi iterasi atau penelusuran elemen pada struktur data queue menggunakan Iterator dalam bahasa Java. Queue dibuat menggunakan LinkedList dan menyimpan beberapa data bertipe string, yaitu “praktikum”, “struktur”, “Data”, “Dan”, dan “Algoritma”. Data-data tersebut ditambahkan ke dalam queue menggunakan method add() sesuai urutan masuknya.

Untuk menampilkan seluruh isi queue, program menggunakan objek Iterator yang diperoleh dari method iterator(). Selanjutnya, perulangan while dengan kondisi hasNext() digunakan untuk memeriksa apakah masih terdapat elemen berikutnya dalam queue. Jika ada, method next() akan mengambil dan menampilkan elemen tersebut satu per satu sesuai urutan antrian. Secara keseluruhan, program ini menunjukkan cara melakukan iterasi pada queue untuk mengakses dan menampilkan seluruh elemen tanpa menghapus data dari dalam antrian.

BAB III

KESIMPULAN

Pada praktikum ini telah dipelajari implementasi struktur data Queue (antrian) dalam bahasa Java menggunakan dua pendekatan, yaitu melalui LinkedList dan array. Queue bekerja dengan prinsip FIFO (First In First Out), di mana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Berbagai operasi dasar queue telah diimplementasikan, seperti menambah data (enqueue/add), menghapus data (dequeue/remove), melihat elemen terdepan (front/peek), melihat elemen terakhir (rear), serta menampilkan seluruh isi antrian (display/iterasi).

Selain operasi dasar, praktikum ini juga menunjukkan penerapan queue pada beberapa kasus lain, seperti membalik urutan data queue menggunakan stack serta melakukan iterasi pada queue menggunakan iterator. Implementasi queue menggunakan array memperkenalkan konsep circular queue agar penggunaan memori lebih efisien, sedangkan implementasi menggunakan LinkedList memberikan kemudahan dalam pengelolaan data secara dinamis. Secara keseluruhan, praktikum ini memberikan pemahaman mengenai konsep, cara kerja, serta penerapan struktur data queue dalam pemrograman Java.